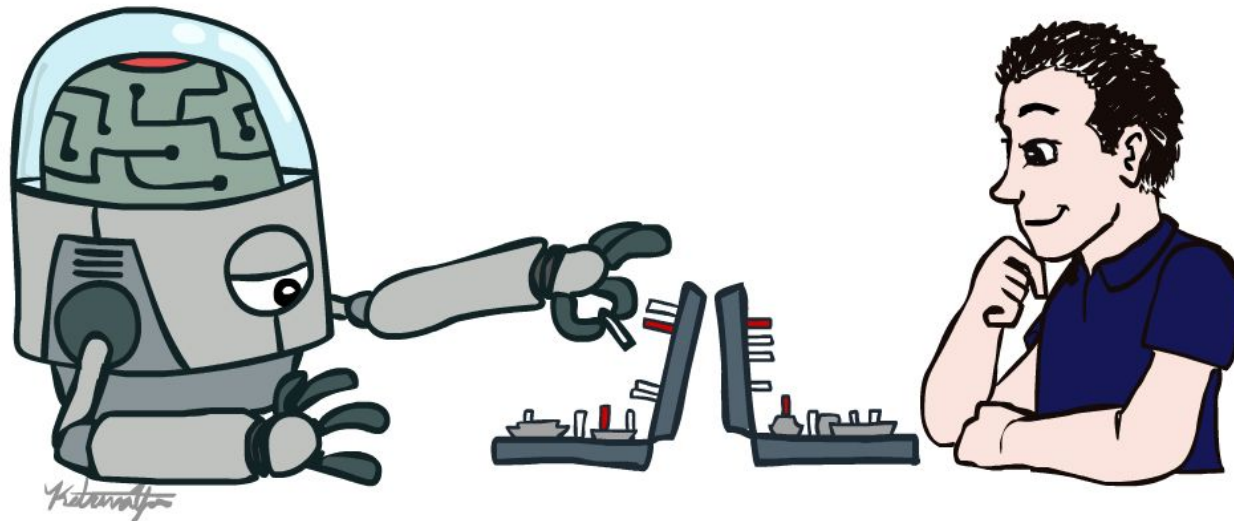


CS-352

Computer Organization and Assembly Language

Lecture 1

Instructor: Aqsa Safdar



Course administration

- **Instructor:** Aqsa Safdar (Lecturer, Deptt of Computer Science, University of Wah, Wah Cantt).
- **Contact Information:**
aqsa.safdar@uow.edu.pk



For queries related to course(Assignments/Projects) just leave an email at

aqsa.safdar@uow.edu.pk

during office timings 08:00AM - 04:00 PM

Evaluation Criteria (2+1 Credit hrs)

Assessment	%Age
Assignments/ Quizzes	$10 + 15\% = 25\%$
Mid Term Exam	30%
Final Term Exam	45%
Total	100

Assignments and Quizzes



Assignments

- Assignments will be due at the beginning of the class. Under normal circumstances, late work will not be accepted.
- Students are expected to do their own assignments.
- Plagiarism will be observed strictly.



Quizzes

- Oral quizzes to check the understanding of previous lecture before starting the next.
- Announced quizzes to assess the understanding of taught topics.

Few Things to Remember

- Show sensibility in your conduct.
- Learning should be your primary objective.
- Attendance will be marked within 10 minutes at the start of the class.
- 80% of your attendance is mandatory.
- **Zero tolerance policy** on attendance, discipline in class during lectures.
- Assignments must be submitted on time, no late submissions.
- In case of copied assignment both parties will be given **zero**.
- Don't miss your Classes, Quizzes, Presentations, Assignments and Projects/Presentations.
- You all need to be a good human being.



Course Objectives

The key objectives of this course include:

- Understanding Computer System Organization
- Assembly Language Programming (MIPS)
- Processor Datapath & ALU Design
- Processor Architectures & Performance
- Memory Hierarchy & System Performance

Course Overview

- Foundations & ISA
- Assembly Programming
- Digital Logic & ALU Design
- Processor Datapath & Control (Verilog HDL)
- Pipelining & Hazards
- Memory Hierarchy & Final Project



OBE

CLO Covered in this Lecture:

CLO1: Explain the basic organization of a computer system, instruction set architecture (ISA), and processor performance metrics to understand how hardware executes programs.

Why Study COAL?

- Understand how software interacts with hardware
- Bridge the gap between high-level programming and machine execution
- Core knowledge for system-level understanding
- AI works faster when the hardware is powerful and efficient.
- Cybersecurity requires low-level system understanding
- Embedded systems demand processor-level programming
- Analyze execution time and CPI
- Understand pipeline and cache performance
- Write efficient, optimized programs using MIPS / Assembly programming
- Verilog HDL design
- Processor data path & control implementation

Reasons to Study COAL

- 1. Understand how a computer works internally** (CPU, memory, registers).
- 2. Learn how instructions are executed** inside the processor.
- 3. Improve problem-solving skills** at hardware level.
- 4. Optimize performance** by understanding memory and processing.
- 5. Build strong foundation** for advanced subjects like Operating Systems and Computer Architecture.

Real life uses of Assembly Language

- Used in embedded systems like **ATMs, traffic lights, and home appliances**
- Used in operating systems for low-level tasks
- Used to write device drivers (keyboard, printer, graphics card)
- Used in game development for speed optimization
- Used in cybersecurity for malware analysis and reverse engineering
- Used in microcontrollers and small hardware devices
- Used where high speed and direct hardware control are required

Why Assembly is Used for High-Speed Applications?

- We talk about **speed in Assembly** because:
- Assembly language is **very close to machine code**.
- It gives **direct control over CPU and registers**.
- There is **no extra layer (like compiler overhead)** as in high-level languages.
- Programs written in Assembly use **less memory**.
- That's why Assembly is used where **high speed and performance are required**, such as:
 - Real-time systems (traffic control, medical devices)
 - Game engines (graphics processing)
 - Embedded systems
 - Device drivers

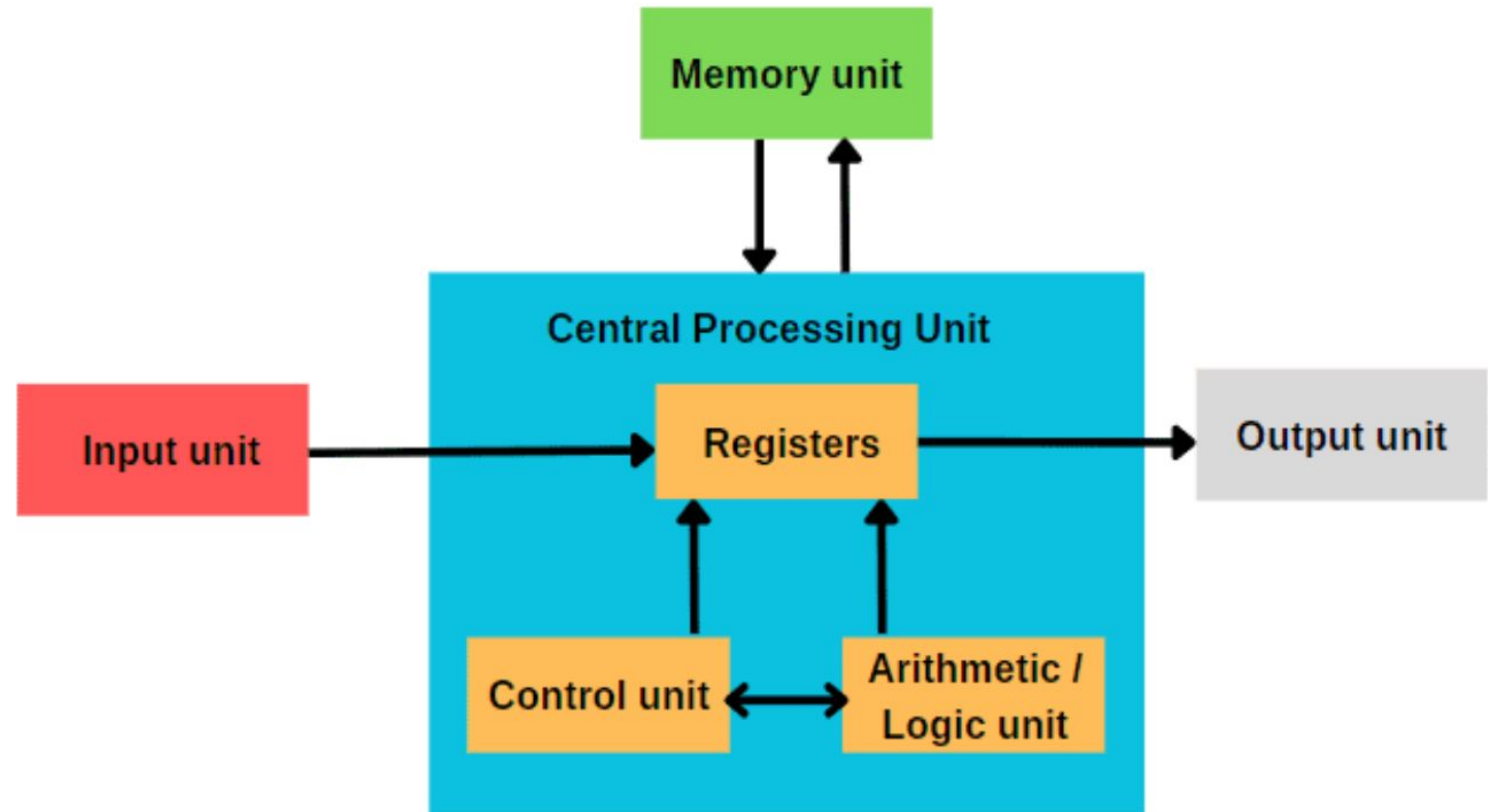
Lecture Outline

- Components of a Computer System:
 - Input Unit
 - Output Unit
 - CPU
 - Memory
- Definition of Computer Organization vs Computer Architecture
- Functional Units
- Von Neumann Architecture
- Performance Metrics (Execution Time, CPI)

Components of a Computer System

Components of a Computer System

- Input Unit
- Output Unit
- CPU
- Memory



Input/output Devices

- **Input Unit:** Takes data from the user and sends it to the CPU.
- **Registers:** Store data temporarily inside the CPU (very fast).
- **Control Unit (CU):** Controls and manages all operations of the CPU.
- **ALU (Arithmetic Logic Unit):** Performs calculations and logical operations.
- **Memory Unit:** Stores data and instructions.
- **Output Unit:** Displays the final result to the user.

Work Flow :

- **Input Unit → Registers → Control Unit → ALU → Memory Unit → Registers (if needed) → Output Unit**

Input Unit

- Accepts data and instructions
- Converts input into binary form

Examples:

- Keyboard
- Mouse
- Scanner
- Microphone

Example:

- When typing:
- $5 + 3$
- Keyboard converts keystrokes into binary signals.



Output Unit

- Converts processed binary results into human-readable form

Examples:

- Monitor
- Printer
- Speakers

Example:

- After computation:
 - 8 displayed on screen.



Screen



Plotter



Printer



Spekar



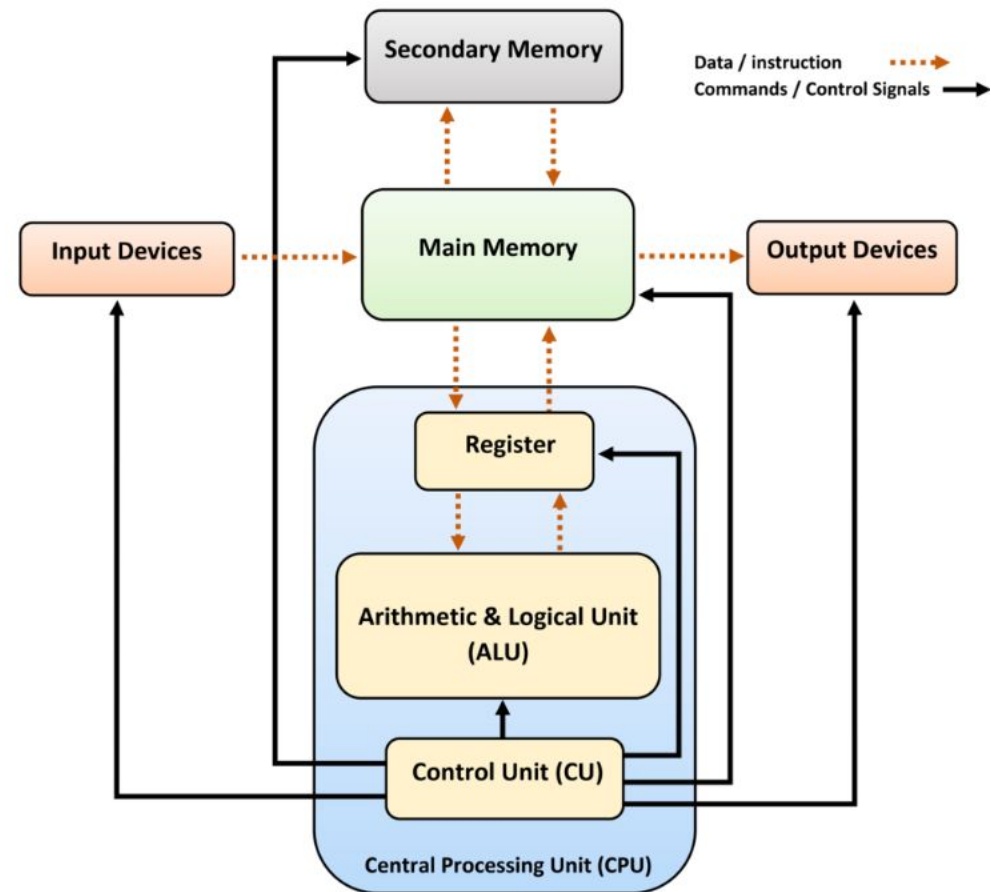
Head phone



Earphone

Central Processing Unit (CPU)

- The CPU is the brain of the computer.
- It consists of:
 - Control Unit (CU)
 - Arithmetic Logic Unit (ALU)
 - Registers



Functional Units of CPU

Control Unit (CU)

- Directs operations
- Sends control signals
- Manages instruction execution

ALU

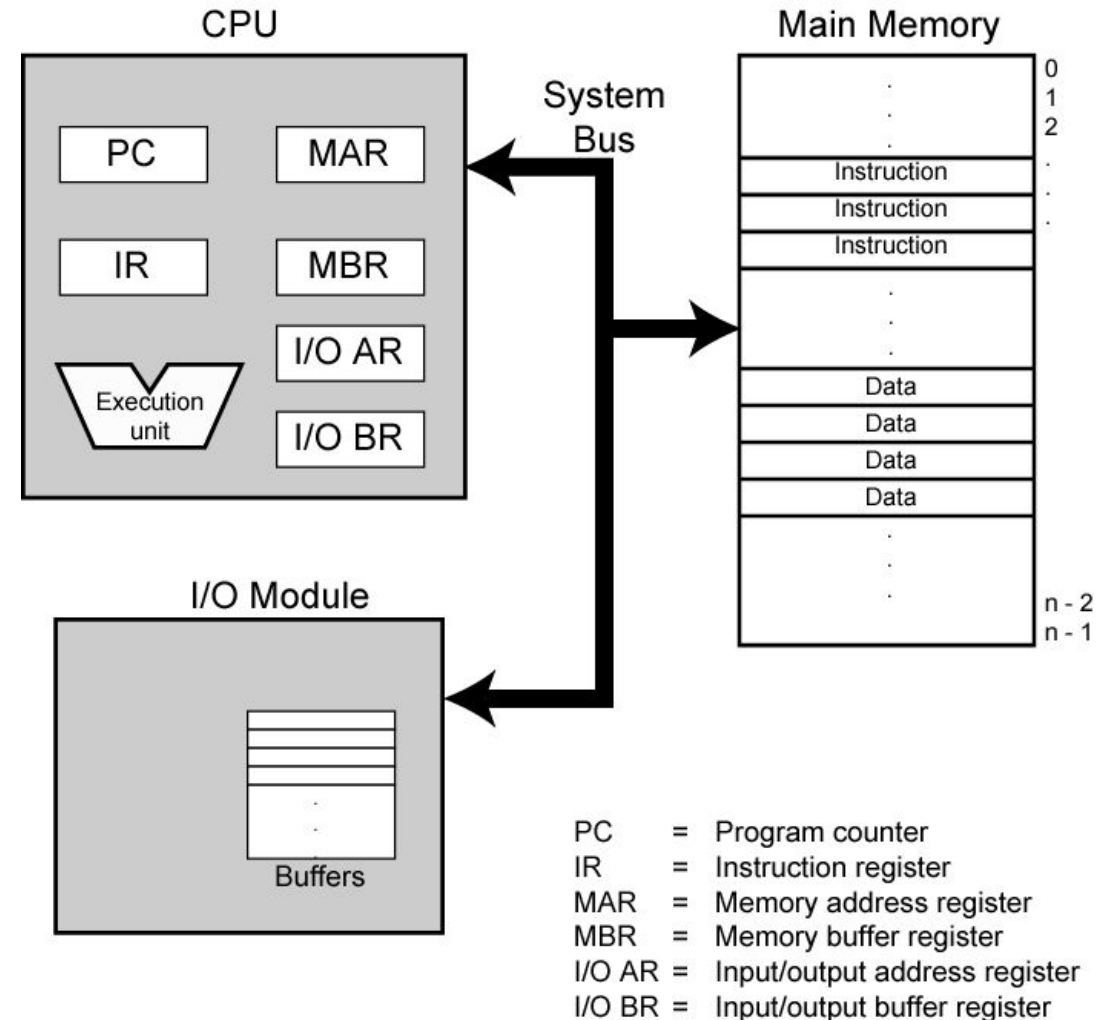
- Performs arithmetic (+, -)
- Performs logical (AND, OR)

Registers

- Small, fast storage

Examples:

- PC (Program Counter)
- IR (Instruction Register)



Memory Unit

Stores:

- Instructions
- Data

Types:

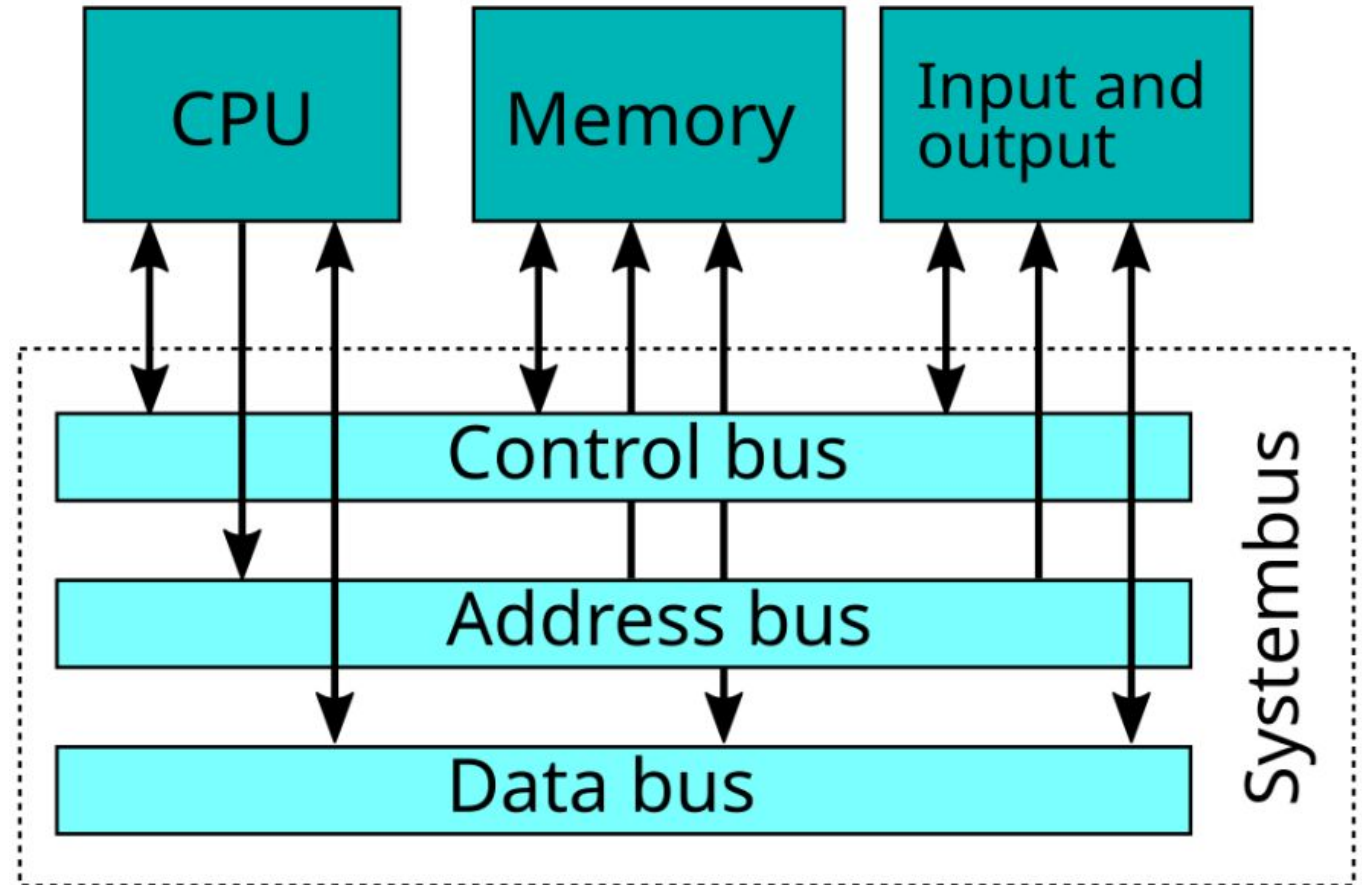
- RAM
- ROM
- Cache

Example:

- Program stored in memory:
 - add \$t0, \$t1, \$t2

Buses

- Data is transmitted from one part of a computer to another, connecting all major internal components to the CPU and memory, by the means of Buses. Types:
 - **Data Bus:** It carries data among the memory unit, the I/O devices, and the processor.
 - **Address Bus:** It carries the address of data (not the actual data) between memory and processor.
 - **Control Bus:** It carries control commands from the CPU (and status signals from other devices) in order to control and coordinate all the activities within the computer.



Computer Architecture

• Computer Architecture Refers to those features of a computer system that are visible to the programmer meaning, things that affect how you write program.

- **Examples:**

- The **instruction set** (what operations the CPU can perform — e.g., add, subtract, multiply)
- The **number of bits** used to represent data (8-bit, 16-bit, 32-bit, 64-bit)
- The **data representation** (how 1's and 0's are used)
- The **addressing modes** (how the CPU accesses data in memory)

- When you write:

```
int a = b + c;
```

Architecture defines:

- Which instruction performs addition
- Which registers are used
- How operands are accessed

Computer Architecture

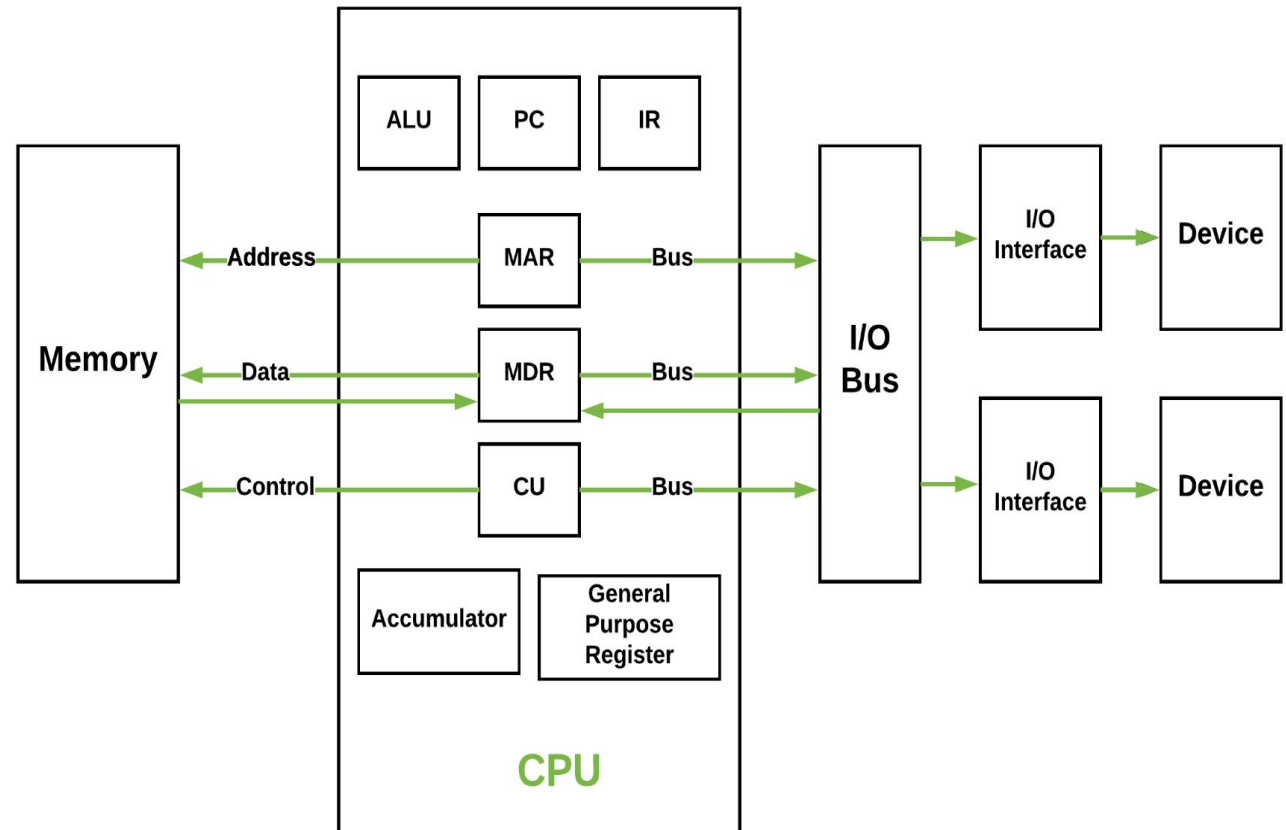
- **ISA** is the set of instructions, registers, and data formats that define how software communicates with hardware.
- **Data types** specify the kind and size of data a processor can store and manipulate (e.g., integer, float, character).
- **Addressing modes** define how an instruction specifies the location of its operands in memory or registers.
- **Registers** are small, high-speed storage locations inside the CPU used to hold data and instructions temporarily.
- **Memory model** describes how data and instructions are organized and accessed within the system's memory structure.

Computer Organization

Computer Organization refers to how hardware components are arranged and interconnected to implement the architecture. **Focus on *how the computer is built?***

Includes:

- Datapath design
- Control signals
- ALU structure
- Pipeline implementation



Computer Organization

- **Datapath design** refers to the arrangement of hardware components (ALU, registers, multiplexers, buses) that move and process data inside the CPU.
- **Control signals** are electrical signals generated by the control unit to direct the operation of data path components during instruction execution.
- **ALU structure** defines the internal hardware organization of the Arithmetic Logic Unit that performs arithmetic and logical operations.
- **Pipeline implementation** is a processor design technique that divides instruction execution into stages to execute multiple instructions simultaneously for improved performance.

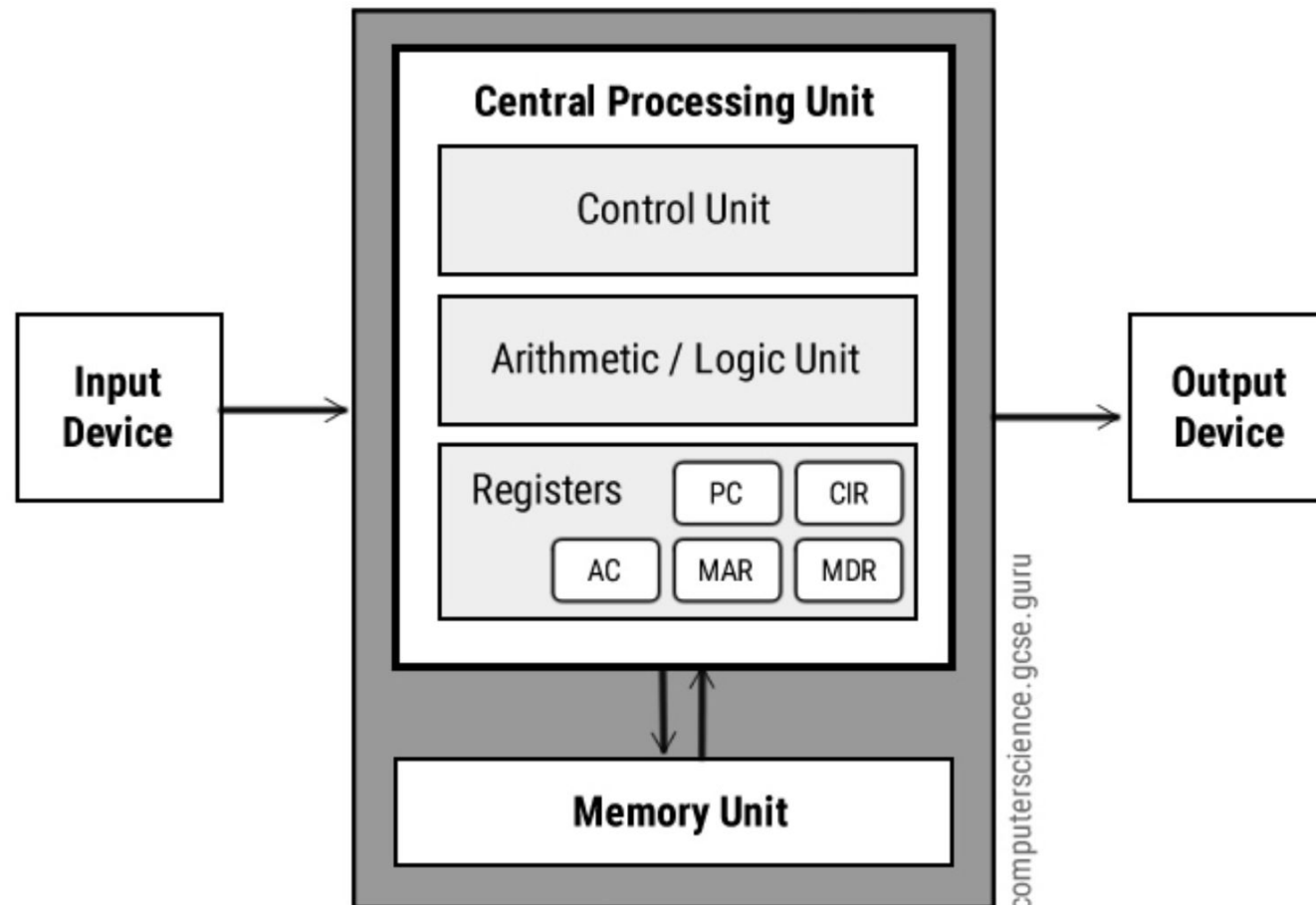
Von Neumann Architecture

- It uses **single memory** to store both the program instructions and the data.
- The CPU fetches instructions and data from the same place, one after another.
- This design is simpler and used in most traditional computers.

Features:

- Stored program concept
- Shared bus
- Sequential execution

Von Neumann Architecture



Registers

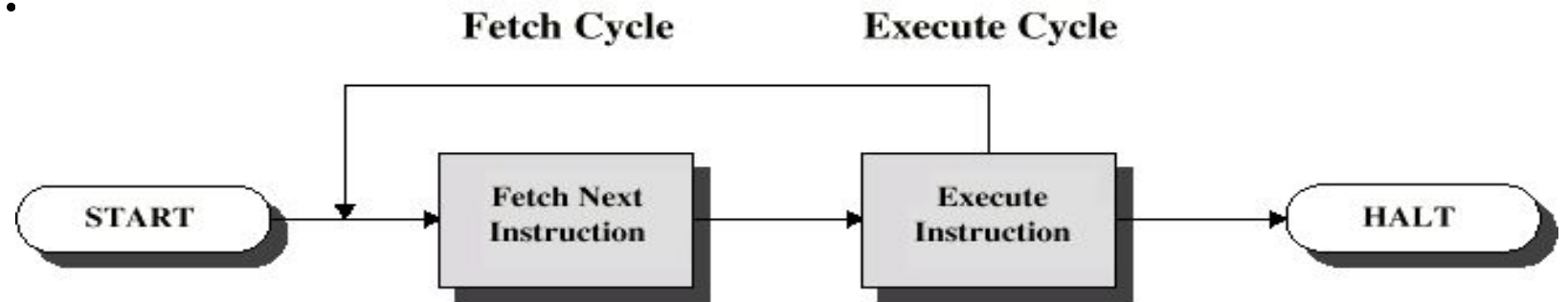
- **Accumulator:** Stores the results of calculations made by ALU.
- **Program Counter (PC):** Keeps track of the memory location of the next instructions to be dealt with. The PC then passes this next address to Memory Address Register (MAR).
- **Memory Address Register (MAR):** It stores the memory locations of instructions that need to be fetched from memory or stored into memory.
- **Memory Data Register (MDR):** It stores instructions fetched from memory or any data that is to be transferred to, and stored in, memory.
- **Current Instruction Register (CIR):** It stores the most recently fetched instructions while it is waiting to be coded and executed.
- **Instruction Buffer Register (IBR):** The instruction that is not to be executed immediately is placed in the instruction buffer register IBR.

Stored Program Concept

Program is stored in memory like data.

Execution cycle:

- Fetch
- Decode
- Execute

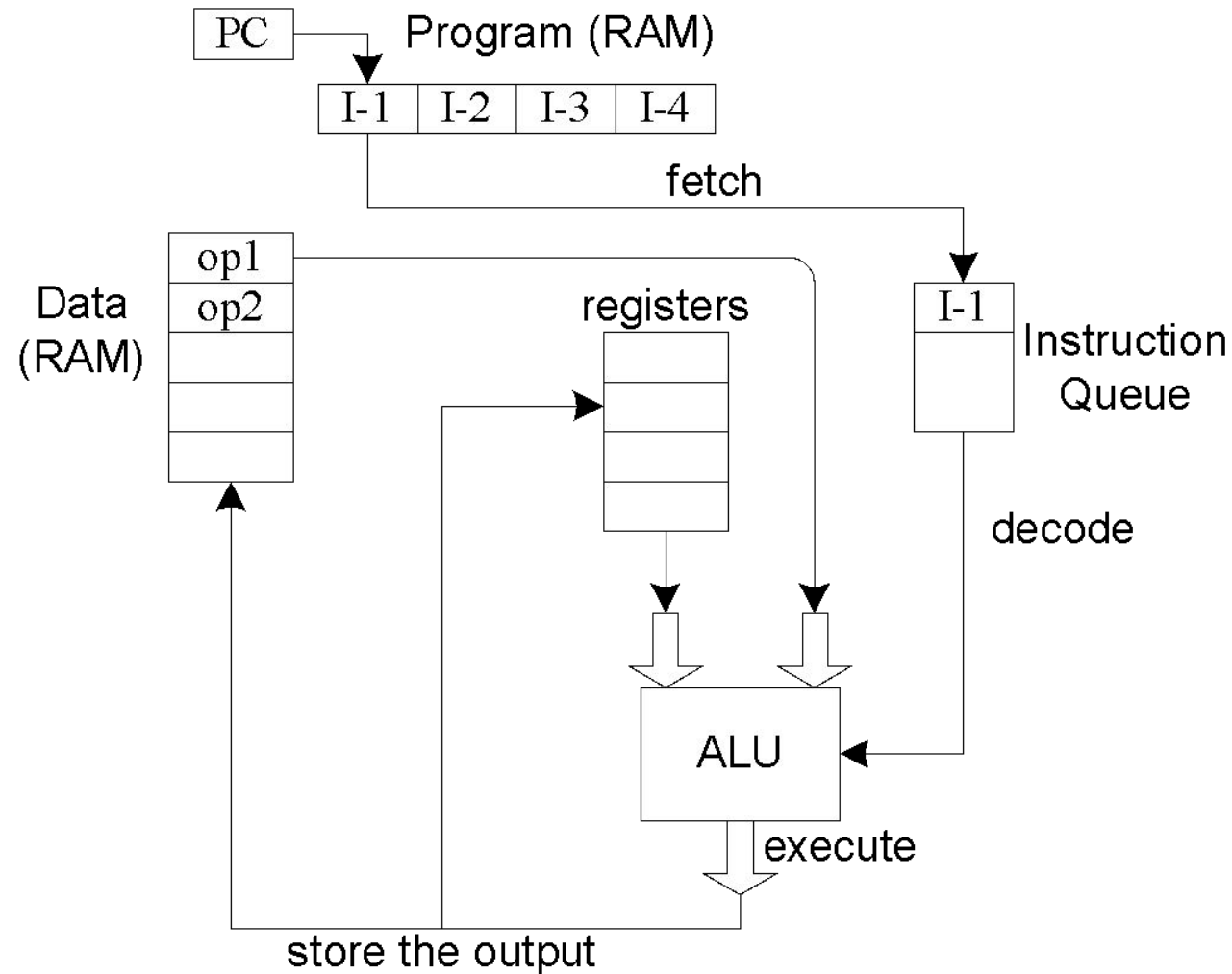


This is called the **Instruction Cycle**.

Stages of Fetch decode cycle

- **Fetch** is the step where the CPU retrieves the next instruction from memory using the Program Counter (PC).
- **Decode** is the step where the CPU interprets the fetched instruction to determine the required operation and operands.
- **Execute** is the step where the CPU performs the operation specified by the instruction using the ALU or other functional units.

Instruction Execution Cycle



Step 1: Fetch

What Happens?

- The **Program Counter (PC)** holds the address of the next instruction.
- The CPU fetches instruction **I-1, I-2, I-3, I-4** from Program RAM.
- The fetched instruction goes to the **Instruction Queue**.

Example:

- If PC = 1000
Instruction at 1000 is:
- add \$t0, \$t1, \$t2
- This instruction is fetched.

Step 2: Decode:

What Happens?

- The instruction moves from the Instruction Queue.
- CPU decodes it:
 - Identifies operation (add)
 - Identifies operands (\$t1, \$t2)
- In the diagram:
 - Instruction Queue → decode → Registers
- The CPU reads operand values from **registers**.

Step 3: Execute

What Happens?

- Operands go to the ALU
- ALU performs the operation
 - In the diagram: Registers $\rightarrow$ ALU $\rightarrow$ execute

Example:

- If:
 - $\$t1 = 5$
 - $\$t2 = 3$
- ALU performs:
 - $5 + 3 = 8$

Step 4: Store Result

- After execution:
- Result is stored back to:
 - A register
 - OR
 - Data Memory (RAM)
- In the diagram: ALU → store the output → Data RAM or Registers

Data Memory

- Data (RAM)
 - op1
 - op2
- Stores:
 - Variables
 - Arrays
 - Operands
- If instruction is:
 - `lw $t0, 0($t1)` # load data from memory to registers
- Data is fetched from Data RAM.

Contd...

Registers:

- Temporarily store operands
- Very fast storage inside CPU

Example registers:

- \$t0
- \$t1
- \$s0
- PC

ALU

- Performs arithmetic operations
- Performs logical operations
- Produces result

Performance Metrics

Why measure performance?

- Compare processors
- Optimize systems
- Improve efficiency

Execution Time

Execution Time = Total time to run a program

Two types:

- Elapsed Time (Wall-clock)
- CPU Time

Example:

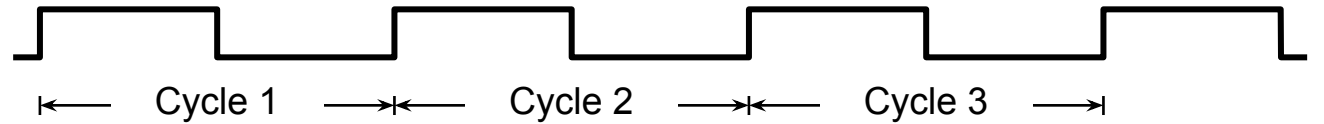
- If a program takes 5 seconds $\rightarrow$ Execution time = 5 sec.

CPU Performance

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{CCT}$$

Where:

- IC= Instruction Count
- CPI = Cycles Per Instruction need
- CCT= Clock Cycle Time



CPI (Cycles Per Instruction)

CPI tells how many clock cycles each instruction needs.

Example:

IC = 1000

CPI = 2

Clock cycle = 1 ns

$$CPU \ Time = 1000 \times 2 \times 1ns = 2000ns$$

Clock Rate vs Clock Cycle

- **Clock rate** tells how many clock cycles a CPU performs in one second. It is usually measured in Hz, MHz, or GHz.

Example:

3 GHz = 3 billion cycles per second

$$3 \text{ GHz} = 3 \times 10^9 \text{ cycles/second}$$

- **Clock cycle time** is the time required to complete one clock cycle.

Formula:

$$\text{Clock Cycle Time} = \frac{1}{\text{Clock Rate}}$$

So if the clock rate increases, the time for one cycle decreases.

Clock Cycle

3. Example

If the processor has a clock rate of 3 GHz:

$$\begin{aligned} \text{Clock Cycle Time} &= \frac{1}{3 \times 10^9} \\ &= 0.33 \text{ ns} \end{aligned}$$

This means one clock cycle takes about 0.33 nanoseconds.

Relationship

- Higher clock rate → more cycles per second
- More cycles → instructions processed faster
- Shorter clock cycle time → faster execution
- So:
- Higher clock rate → shorter cycle time → faster execution

Task

Processor A:

- $\text{CPI} = 1$
- $\text{Clock} = 2 \text{ GHz}$

Processor B:

- $\text{CPI} = 2$
- $\text{Clock} = 3 \text{ GHz}$

Calculate which is faster?

Summary

- Architecture vs Organization
- Functional Units
- Von Neumann Model
- Stored Program Concept
- Execution Time
- CPI
- Performance Formula